期末專題技術報告：Small-C 互動式解譯器

林煜博

Spring 2026

Contents

1 摘要

本專題實作一個 Small-C 語言的互動式解譯器（Interactive Interpreter），以純 Python 3 撰寫，不依賴任何第三方套件。解譯器採用經典的四階段管線架構：前處理（Preprocessor）→ 詞法分析（Lexer）→ 語法分析（Parser）→ 樹狀走訪直譯（Tree-Walking Interpreter），並提供完整的 REPL（Read-Eval-Print Loop）操作介面，支援程式的互動輸入、載入、編輯、執行與除錯追蹤。

本報告詳細說明各模組的設計決策與實作細節、符號表與記憶體模型的資料結構設計、測試策略與結果分析，以及開發過程中遭遇的技術困難與解決方案。

2 系統架構設計說明

2.1 模組劃分

整個解譯器共分為七個 Python 模組，各司其職：

- **`main.py`**：程式進入點，建立並啟動 `REPL` 物件。
- **`lexer.py`**：詞法分析層。`preprocess()` 負責展開無參數 `#define` 巨集；`Lexer.tokenize()` 逐字元掃描，將原始碼轉為 `Token` 串流供 Parser 消費。
- **`parser.py`**：語法分析層，包含兩大部分：上半部定義所有 AST 節點類別（`Expr`、`Stmt` 及其子類別），下半部實作遞迴下降 Parser，將 Token 串流轉為抽象語法樹（AST）。
- **`interpreter.py`**：直譯執行層。走訪 Parser 產生的 AST，以 Python 例外實作 `break` ／ `continue` ／ `return` 控制流程，支援完整程式執行（`execute`）與互動片段執行（`execute_interactive`）兩種模式。
- **`memory.py`**：記憶體模型。以固定大小的整數陣列模擬線性記憶體空間，提供 bump allocator 進行配置與釋放；模組層級的 `int32()` 函式負責 32 位元整數截斷，供其他模組共用。
- **`symtable.py`**：符號表。以作用域堆疊（`list[dict]`）管理變數名稱與記憶體位址的對映，實現 C 語言的區域／全域作用域與變數遮蔽行為。
- **`builtins_funcs.py`**：內建函式庫。以分派字典實作全部 22 個內建函式，包含輸出入（`printf`、`scanf`）、字串處理（`strlen`、`strcpy` 等）、數學運算（`abs`、`sqrt` 等）及工具函式（`atoi`、`itoa` 等）。
- **`repl.py`**：互動介面。`ReplInputCollector` 追蹤大括號深度與字串狀態，收集跨行輸入；`REPL` 以分派字典處理 16 個環境指令，並處理互動模式下的 `else` 跨行銜接問題。

圖一為各模組的相依關係圖：

由圖中可見各模組的依賴關係如下：

- `repl.py` 是整合核心，依賴 `interpreter.py` 執行程式。
- `interpreter.py` 依賴 `parser.py`（取得 AST）、`memory.py`（記憶體讀寫）、`symtable.py`（符號查詢）與 `builtins_funcs.py`（內建函式）。
- `parser.py` 依賴 `lexer.py` 取得 Token 串流。
- `memory.py` 被 `symtable.py` 與 `builtins_funcs.py` 共用，是底層的共享資源。

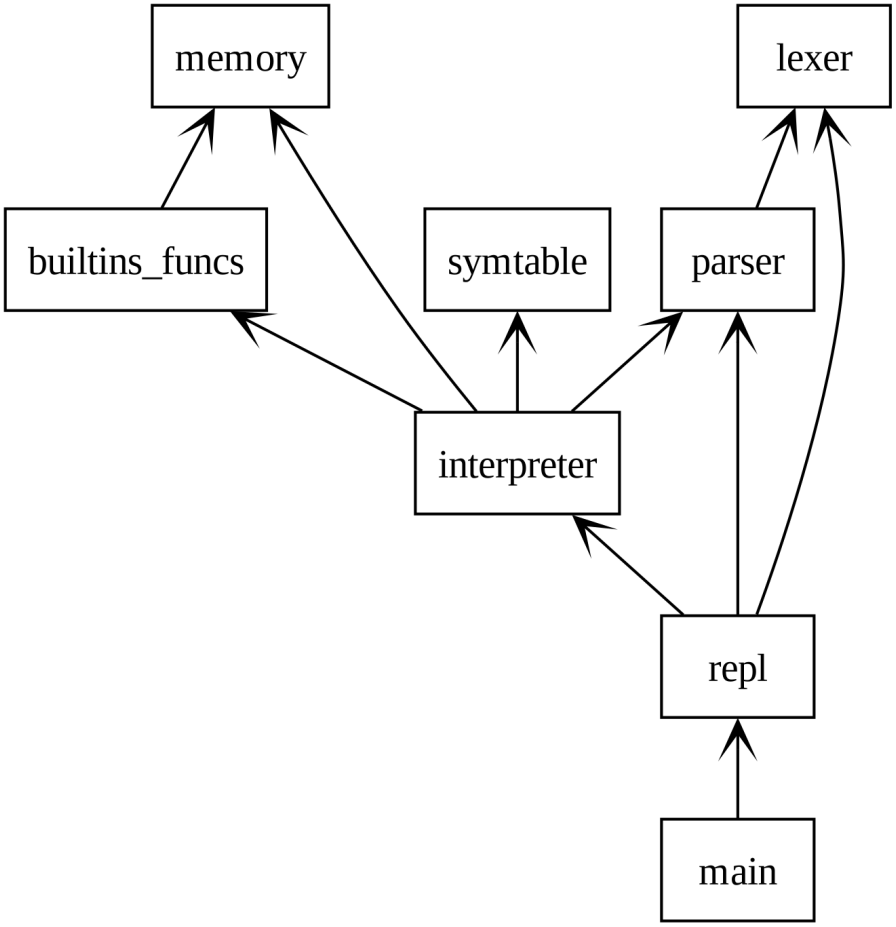

圖 1: 圖一：模組相依關係圖

2.2 資料流程

原始碼從輸入到輸出，嚴格單向流動：

```
使用者輸入（REPL）或 LOAD 指令
        │
        ▼
  preprocess()          ── #define 巨集展開
        │
        ▼
  Lexer.tokenize()      ── 字元掃描 → Token 串流
        │
        ▼
  Parser.parse()        ── 遞迴下降 → AST
        │
        ▼
  Interpreter.execute() ── 走訪 AST → 產生輸出
        │
        ▼
   stdout 輸出
```

互動模式（直接在提示符下輸入語句）走 `execute_interactive()` 路徑，跳過「必須有 `main()`」的限制；`RUN` 指令走 `execute()` 路徑，要求程式含有 `main()` 函式作為進入點。

2.3 類別結構

圖二與圖三為各模組中主要類別與 AST 節點的完整類別圖：

AST 節點分為三個層次：

- 頂層節點
  - `Program`：整份程式的根節點，包含所有頂層宣告
  - `FuncDef`：函式定義，含名稱、回傳型別、參數列表與函式本體
- **`Expr`**（運算式）子類別
  - 算術／邏輯：`BinOp`（二元運算）、`UnaryOp`（一元運算）
  - 賦值：`Assignment`（含複合賦值 `+=` 等）
  - 呼叫：`Call`（函式呼叫）
  - 基本值：`Identifier`（識別字）、`Number`（整數）、`Char`（字元）、`StringLiteral`（字串）
  - 指標：`AddressOf`（取址 `&`）、`Deref`（解參考 `*`）、`ArrayAccess`（陣列索引）
- **`Stmt`**（陳述式）子類別
  - 區塊：`Block`（大括號區塊）

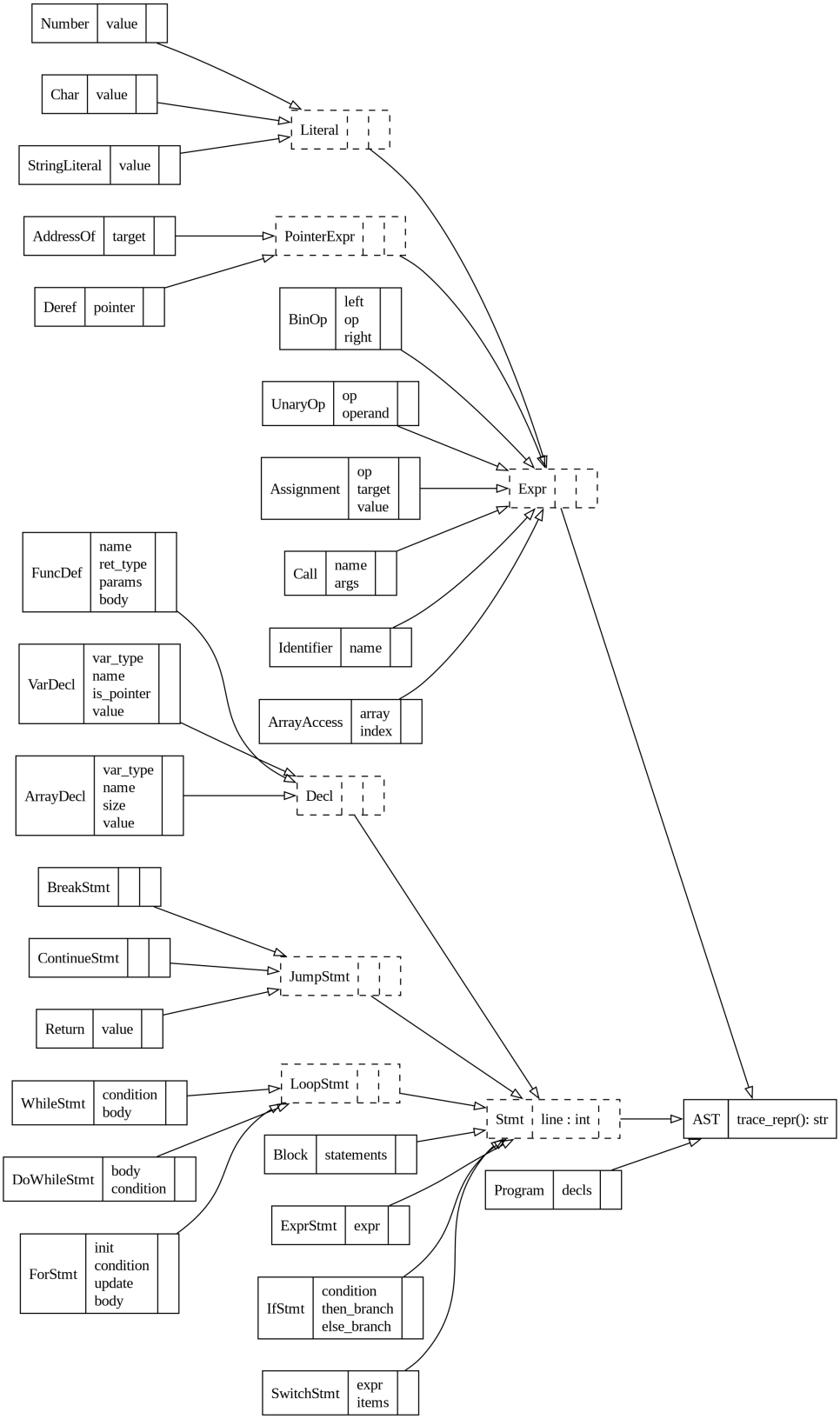

圖 2: 圖二：AST 節點繼承層次

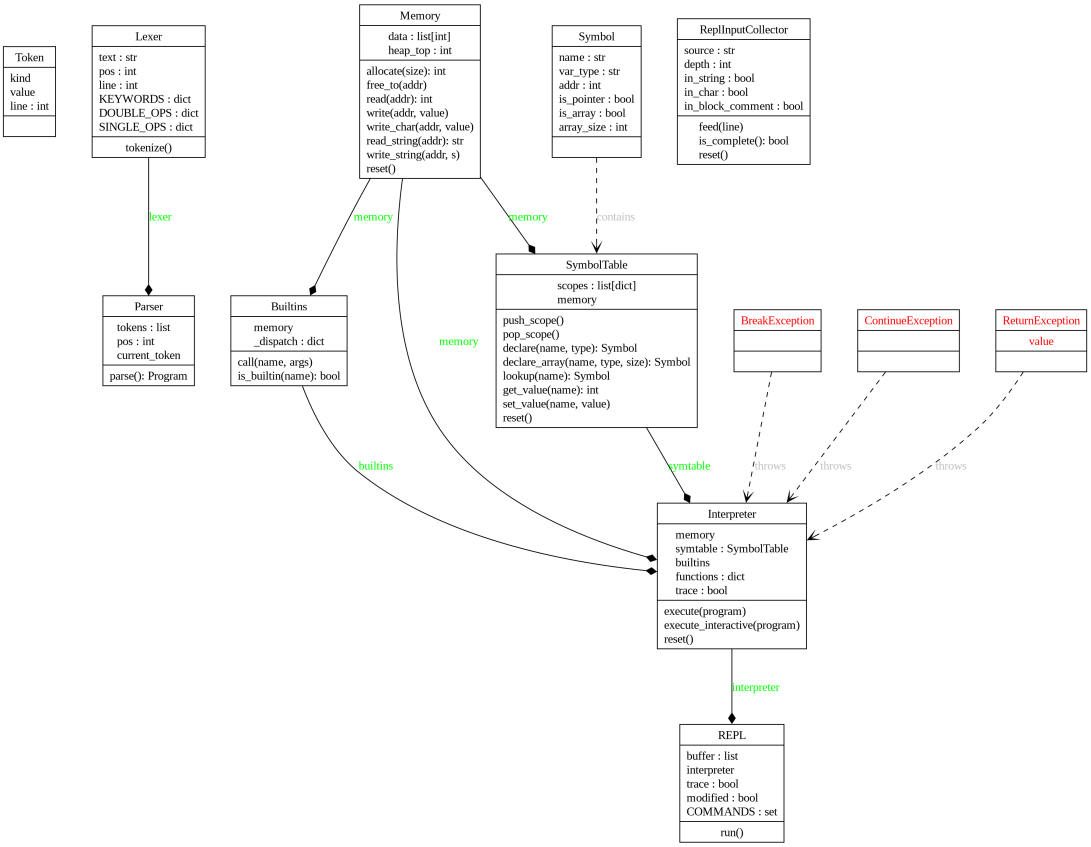

圖 3: 圖三：執行期模組類別圖

- 選擇：`IfStmt`（if/else）、`SwitchStmt`（switch/case）
- 迴圈：`WhileStmt`、`DoWhileStmt`、`ForStmt`
- 跳躍：`BreakStmt`、`ContinueStmt`、`Return`
- 宣告：`VarDecl`（變數宣告）、`ArrayDecl`（陣列宣告）
- 其他：`ExprStmt`（運算式陳述式）